第 18 章：上下文管理器与资源生命周期

一句话概括本章：`with` **把“资源必须在所有离开路径上释放”写成了一条可复用的协议。**

文件、数据库连接、事务、锁、HTTP 客户端和工具会话都存在生命周期。获得资源通常不难，难的是保证正常结束、提前 `return`、块内异常和后续资源获取失败时都不会漏掉清理。只在业务代码末尾写一句 `close()` 并不可靠，因为前面的任何跳转或异常都可能绕过它。

对于 Java 工程师，可以先把 Python 的 `with` 类比为 `try-with-resources`，但两者不能完全等同。Python 不要求对象实现名为 `AutoCloseable` 的接口，而是在运行时使用 `__enter__()` 和 `__exit__()` 协议；`as` 后面的值甚至可以不是管理器对象本身。

完成本章后，你应该能够：

- 解释 `with` 如何保证成功进入的资源一定执行退出逻辑；
- 区分上下文管理器、`__enter__()` 返回值和 `as` 目标；
- 预测正常结束、`return`、块内异常和进入失败时的执行顺序；
- 正确实现类上下文管理器和 `@contextmanager`；
- 用多项 `with` 和 `ExitStack` 逆序清理多个资源；
- 判断谁拥有资源、谁只是借用资源；
- 在资源型生成器提前停止时建立明确的关闭边界。

本章只讨论同步上下文管理器。`async with`、`asynccontextmanager` 和 `AsyncExitStack` 会在后续异步章节中展开。

18.1 先建立 Java → Python 心智模型

Java 和 Python 解决的是同一个工程问题：资源一旦成功取得，退出使用区间时就必须清理。不过，两门语言的协议和异常语义不同。

关注点	Java `try-with-resources`	Python `with`
资源协议	`AutoCloseable.close()`	`__enter__()` 与 `__exit__()`
协议检查	编译期要求资源类型可关闭	运行时查找上下文管理协议
块内变量	通常就是声明的资源对象	`as` 绑定 `__enter__()` 的返回值

正常退出	自动调用 `close()`	调用 `__exit__(None, None, None)`
块内异常	关闭后继续传播，关闭异常有 Java 的 suppressed 规则	`__exit__()` 能观察异常， 并可通过真值抑制它
多资源	从后向前关闭	从后向前调用 `__exit__()`
块级作用域	资源变量离开 `try` 后不可见	`as` 名字通常仍可见，但资源可能已经关闭

最重要的两个不等价点是：

1. Python 的管理器和块内句柄可以是两个对象。管理器负责生命周期，句柄只暴露允许使用的能力。
2. Python 的 `__exit__()` 返回值会参与异常控制流。误返回一个真值，可能把真正的业务异常吞掉。

因此，看到下面的代码时，不要只理解为“最后自动执行 `close()`”：

```
with make_session() as session:
    session.call_tool()
```

更准确的理解是：`make_session()` 先产生管理器，管理器成功进入后把一个值交给 `session`，离开缩进块时再由同一个管理器处理退出。

18.2 `with` 是协议，不是文件专用语法

一个类只要提供 `__enter__()` 和 `__exit__()`，就可以参与同步 `with` 协议：

```
class Session:
    def __enter__(self) -> "Session":
        print("打开会话")
        return self

    def call(self) -> str:
        return "调用完成"

    def __exit__(self, exc_type, exc_value, traceback) -> bool:
        print("关闭会话")
        return False


with Session() as session:
```

README × | readme × | README × | README × | README × | RE
7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-34018e1e0d69d85a3c656d7be7e8.html
... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

```
with Session() as session:
    print(session.call())
```

输出顺序是：

```
打开会话
调用完成
关闭会话
```

这里有三个容易混淆的角色：

- `Session()` 创建的对象是 **context manager（上下文管理器）**；
- `__enter__()` 的返回值才会绑定给 `as session`；
- `__exit__()` 属于管理器，负责离开本次上下文。

上例的 `__enter__()` 返回 `self`，所以管理器和块内值恰好是同一个对象。协议并没有要求这样做。
真实 SDK 可以让管理器保留关闭权限，只返回一个受限句柄：

```
manager = ToolSessionManager()
with manager as handle:
    handle.call("search")

assert handle is not manager
```

这可以先类比为 Java 所有者把受限接口借给业务代码，但不能完全等同于 Java 的资源变量：Python 的 `as` 明确接收进入方法的返回值，而不是固定接收管理器本身。

18.3 一次 `with` 到底怎样执行

单个 `with` 的核心过程可以概括为：

```
计算上下文表达式
    ↓
得到管理器并调用 __enter__()
    ↓
把返回值绑定给 as 目标
    ↓
执行 with 块
```

README × | readme × | README × | README × | README × | REA
vf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-34018e1e0d69d85a3c656d7be7e8.html
目关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

18.3 一次 `with` 到底怎样执行

单个 `with` 的核心过程可以概括为：

```
计算上下文表达式
    ↓
得到管理器并调用 __enter__()
    ↓
把返回值绑定给 as 目标
    ↓
执行 with 块
    ↓
调用同一管理器的 __exit__()
```

官方语义可以用下面的概念代码展开。实际解释器会按特殊方法规则查找协议方法，但生命周期顺序与这里一致：

```
import sys


manager = EXPRESSION
enter = manager.__enter__
exit_method = manager.__exit__
value = enter()
hit_exception = False

try:
    TARGET = value
    SUITE
except BaseException:
    hit_exception = True
    if not exit_method(*sys.exc_info()):
        raise
finally:
    if not hit_exception:
        exit_method(None, None, None)
```

由此可以推导出一条关键规则：**只有 `__enter__()` 正常返回，Python 才承诺为本次 `with` 调用 `__exit__()`。成功进入之后，即使 `as` 目标绑定失败，也会进入退出流程。**

常见路径如下：

离开路径	是否调用本次 `__exit__()`	传入的异常信息
运行到块末尾	是	三个 `None`
块内 `return`	是	三个 `None`
块内 `break` / `continue`	是	三个 `None`
块内抛异常	是	异常类型、异常对象、traceback
计算上下文表达式时失败	否	尚未得到管理器
`__enter__()` 抛异常	否	尚未成功进入

`return` 不会抢在清理之前离开函数。Python 会先准备返回值，再调用 `__exit__(None, None, None)`，清理成功后调用方才收到返回值。如果退出逻辑又抛出异常，调用方不会正常收到原返回值。

18.4 `__exit__()` 如何看到并决定异常命运

带类型标注的典型签名如下：

```
from types import TracebackType


def __exit__(
    self,
    exc_type: type[BaseException] | None,
    exc_value: BaseException | None,
    traceback: TracebackType | None,
) -> bool:
    self.close()
    return False
```

正常结束、`return`、`break` 和 `continue` 都不是异常，因此三个参数都是 `None`。如果块内抛出 `ToolCallError("timeout")`：

- `exc_type` 是 `ToolCallError` 类；
- `exc_value` 是消息为 `"timeout"` 的异常对象；

README | readme | README | README | README | READM
72502x6dg8fpjj0sff40000gp/T/crossnote-f-34018e1e0d69d85a3c656d7be7e8.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

正常结束、`return`、`break` 和 `continue` 都不是异常，因此三个参数都是 `None`。如果块内抛出 `ToolCallError("timeout")`：

- `exc_type` 是 `ToolCallError` 类；
- `exc_value` 是消息为 `"timeout"` 的异常对象；
- `traceback` 记录异常经过的调用栈。

退出方法的返回值只影响正在传播的块内异常：

`__exit__()` 的结果	块内异常怎样处理
假值，例如 `False` 或 `None`	原异常继续向外传播
任意真值，例如 `True`、`1`、非空字符串	原异常被抑制，继续执行 `with` 后面的语句
`__exit__()` 自己抛出新异常	新异常向外传播，原异常成为异常上下文

普通资源管理器通常只负责清理，不负责把失败变成成功，因此应返回 `False`，或者不写 `return` 让函数自然返回 `None`。

不要顺手返回清理函数的结果：

```
# 错误：若 close_and_report() 返回 "closed"，块内异常会被隐藏。
def __exit__(self, exc_type, exc_value, traceback):
    return self.close_and_report()
```

有意抑制异常并非永远错误。例如，一个明确约定"目标文件不存在时使用默认值"的专用管理器，可以只抑制 `FileNotFoundError`。但这必须是公开 API 语义，而且应严格判断异常类型，不能无条件返回真值。

18.5 为什么 `__enter__()` 失败必须自己回滚

真实连接往往分阶段建立：

```
分配 socket → 建立 TLS → 鉴权 → 发布可用会话
```

假设 socket 已分配，而鉴权抛出异常。由于 `__enter__()` 没有正常返回，本次 `with` 不会调用 `__exit__()`。获取逻辑必须回滚自己已经完成的部分初始化：

```
def __enter__(self):
    self._allocate_transport()
    try:
        self._authenticate()
        return self._build_handle()
    except BaseException:
        self._release_transport()
        raise
```

这里捕获 `BaseException` 不是为了把所有失败转成业务结果，而是为了释放已取得的底层资源后原样重抛。这样即使遇到 `KeyboardInterrupt` 等退出信号，也不会遗留已分配的资源。回滚操作最好是幂等的，避免错误路径再次清理时产生第二个故障。

这个边界与 Java 构造资源时的情况相似：如果第二个资源初始化失败，已经成功创建的第一个资源会由外层结构关闭；但第二个对象内部已经完成的部分初始化，仍要由它自己的获取逻辑负责回滚。调用方既不知道完成了哪些步骤，也不应该猜测半初始化对象能否安全执行完整 `close()`。

18.6 `try/finally` 是底层控制流，`with` 是可复用封装

只有一处使用资源时，最基本的安全骨架是：

```
resource = acquire()
try:
    use(resource)
finally:
    release(resource)
```

正常结束、`return` 和异常都会在离开 `try` 时先执行 `finally`。如果 `acquire()` 自己失败，它不在 `try` 范围内，因此仍须按照上一节的原则自行回滚部分初始化。

下面两种写法都不安全：

```
# use() 失败会绕过 release()。
resource = acquire()
use(resource)
release(resource)


# finally 中的 return 会覆盖原返回值或正在传播的异常。
```

```
# finally 中的 return 会覆盖原返回值或正在传播的异常。
try:
    use(resource)
finally:
    return release(resource)
```

Python 3.14 还会对 `finally` 中的 `return`、`break` 或 `continue` 发出 `SyntaxWarning`。工程代码应让 `finally` 专注于清理，不要在其中改写原控制流。

当多处代码都需要同一种生命周期时，再把这段 `try/finally` 封装成类协议。类实现适合以下情况：

- 对象有 `NEW → OPEN → CLOSED` 之类的显式状态；
- 获取过程需要配置、多个辅助方法或失败回滚；
- 管理器要返回与自身不同的借用句柄；
- 退出时要根据异常类型执行明确策略。

所以 `with` 不是 `try/finally` 的替代品。前者把后者表达的清理边界封装成了可组合协议。

18.7 用 `@contextmanager` 表达线性生命周期

如果生命周期可以按"获取 → 使用 → 清理"顺序写清，`contextlib.contextmanager` 能省去类的样板代码：

```
from collections.abc import Generator
from contextlib import contextmanager


@contextmanager
def managed_session() -> Generator[Session]:
    session = Session()
    session.open()
    try:
        yield session
    finally:
        session.close()


with managed_session() as session:
    session.call()
```

README | readme | README | README | README | README
2502x6dg8fpjj0sff40000gp/T/crossnote-f-34018e1e0d69d85a3c656d7be7e8.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

```
with managed_session() as session:
    session.call()
```

装饰器把生成器函数转换成“每次调用都创建一个上下文管理器”的工厂。这里的 `yield` 不是在生产一串业务数据，而是在划分上下文的进入和退出：

1. 调用 `managed_session()` 得到管理器，生成器函数体还没有完整执行；
2. `with` 进入时运行到 `yield`，把 `session` 绑定给 `as` 目标；
3. 生成器暂停，转而执行 `with` 块；
4. 块正常结束时，从 `yield` 后恢复；
5. 块内抛异常时，同一个异常会在暂停的 `yield` 位置重新抛出；
6. `finally` 负责关闭资源，然后生成器结束。

一次上下文生命周期必须 **恰好执行一次 `yield`**。如果生成器正常结束却没有执行 `yield`，就没有进入值；执行第二次 `yield`，退出过程又无法结束。`contextmanager` 会用 `RuntimeError` 拒绝这两种实现。若到达 `yield` 之前的获取代码自己抛出异常，向外传播的则是该获取异常。

异常路径还有一个很隐蔽的坑：

```
@contextmanager
def managed_session():
    session = open_session()
    try:
        yield session
    except BaseException as error:
        log(error)
        raise
    finally:
        session.close()
```

块内异常会回到 `yield` 处。如果 `except` 只记录日志却没有重新 `raise`，生成器会正常结束，`contextmanager` 就会认为异常已被处理。只需要清理时，省略 `except`、只保留 `try/finally` 最稳妥；确实需要记录时，要用裸 `raise` 保留原异常和 traceback。

获取动作如果在到达 `try` 和 `yield` 之前失败，仍然要自己回滚部分初始化。这一点与类管理器的 `__enter__()` 失败完全相同。

18.8 多个资源为什么按相反顺序退出

18.8 多个资源为什么按相反顺序退出

下面两种写法语义相同：

```
with Resource("A") as a, Resource("B") as b, Resource("C") as c:
    use(a, b, c)


with Resource("A") as a:
    with Resource("B") as b:
        with Resource("C") as c:
            use(a, b, c)
```

进入顺序是 `A → B → C`，退出顺序是 `C → B → A`。后取得的资源可能依赖先取得的资源，例如游标依赖连接、事务依赖会话。后进先出可以先拆掉依赖者，再拆基础设施。

如果 B 进入失败，执行结果是：

- B 没有成功进入，所以不会调用 B 的 `__exit__()`；
- B 内部的部分初始化由 B 自己回滚；
- 已成功进入的 A 会退出；
- C 从未开始进入。

这与 Java `try-with-resources` 的多资源展开方式很接近。固定少量资源时，多项 `with` 最容易读。当资源数量只有运行时才知道时，则需要 `ExitStack`。

18.9 `ExitStack`：动态建立退出栈

假设文件列表来自配置，源码无法预先写出 N 层 `with`：

```
from contextlib import ExitStack


with ExitStack() as stack:
    files = [stack.enter_context(open(path, encoding="utf-8")) for path in runtime_paths]
    process(files)
```

`enter_context(manager)` 会立即调用该管理器的 `__enter__()`。只有进入成功，对应的退出方法才会

README × | readme × | README × | README × | README × | READM
72502x6dg8fpjj0sff40000gp/T/crossnote-f-34018e1e0d69d85a3c656d7be7e8.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

`enter_context(manager)` 会立即调用该管理器的 `__enter__()`。只有进入成功，对应的退出方法才会压入栈中；它的返回值则交给调用方使用。离开外层 `with` 时，已登记的方法按后进先出顺序调用。

如果第三个资源进入失败，时间线如下：

```
资源 1 进入成功，退出方法入栈
资源 2 进入成功，退出方法入栈
资源 3 进入失败
    ├─ 资源 3 自己回滚部分初始化
    ├─ ExitStack 退出资源 2
    └─ ExitStack 退出资源 1
```

`ExitStack` 还可以登记普通回调，因此适合把不同清理形式组合在一个动态边界里。它是偏底层的工具，不应取代本来就很清楚的单个 `with`。另外，垃圾回收 `ExitStack` 对象时不会隐式执行已登记的回调；应把它放进 `with`，或由明确所有者调用 `close()`。

`pop_all()` 可以把整组退出责任转移到一个新的 `ExitStack`，但不会当场清理。只有当 API 确实需要跨作用域转移所有权时才使用它，并把新所有者写进接口契约。

18.10 所有权：谁创建，谁负责关闭

“对象有 `close()` 方法”不等于“当前函数有权调用它”。处理资源前，先回答三个问题：谁创建、谁使用、谁关闭。

```
请求处理函数（所有者）
├─ 创建会话并进入 with
├─ 把句柄借给 search_tool（只使用）
├─ 把同一句柄借给 summary_tool（只使用）
└─ 离开 with，统一关闭一次
```

角色	资源从哪里来	可以做什么	关闭责任
所有者	自己创建，或明确接收所有权	使用、转借、决定生命周期	必须在所有路径清理
借用者	通过参数临时获得	在约定范围内使用	默认不能关闭
新所有者	API 明确转移了所有权	接管后续生命周期	从转移点开始负责清理

下面的借用者看似负责，实际会破坏共享会话：

README | readme | README | README | README | READI
z72502x6dg8fpjj0sff40000gp/T/crossnote-f-34018e1e0d69d85a3c656d7be7e8.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

```
def run_tool(client):
    try:
        return client.call()
    finally:
        client.close()  # 第一个工具结束后，其他借用者无法继续使用。
```

正确做法是让真正创建客户端的外层边界统一关闭。Python 的 `with` 不创建块级变量作用域，因此 `as handle` 的名字离开块后通常还存在；但它指向的底层资源已经关闭，继续调用属于生命周期错误。

当函数既允许“自己创建客户端”，又允许“调用方传入共享客户端”时，两条所有权路径必须分开：

```
from contextlib import nullcontext


def execute(client=None):
    scope = Client() if client is None else nullcontext(client)
    with scope as active_client:
        return active_client.call()
```

可以先把 `nullcontext(client)` 理解为“为了统一控制流而创建的空作用域”：进入时返回原对象，退出时什么也不做。它让借用对象可以复用同一段 `with` 代码，但不会擅自接管关闭责任。

18.11 生成器提前停止时，`break` 不等于关闭

资源型生成器可能在第一次消费时打开会话，然后暂停在 `yield`：

```
def stream_results(manager, commands):
    with manager as session:
        for command in commands:
            yield session.call(command)
```

如果消费者自然耗尽生成器，控制流最终会离开内部 `with`。但消费者可能只取第一项：

```
stream = stream_results(manager, commands)
for result in stream:
    consume(result)
    break
```

README × | README × | readme × | README × | README × | README × | REA
z72502x6dg8fpjj0sff40000gp/T/crossnote-f-34018e1e0d69d85a3c656d7be7e8.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

```
# stream 仍引用暂停在 yield 处的生成器，内部资源可能仍然打开。
stream.close()
```

`for` 协议负责调用 `iter()` 、 `next()` 并处理 `StopIteration` 。 `break` 只停止循环，不会承诺对任意迭代器调用 `close()` 。不要把 CPython 在某些情况下很快回收临时生成器的现象当成资源管理契约；只要仍有引用，生成器就可能继续暂停。

对生成器调用 `close()` ，会在它暂停的 `yield` 位置引发 `GeneratorExit` ，使外围的 `finally` 或 `with` 有机会执行清理。也可以用 `closing()` 把所有权边界写进结构：

```
from contextlib import closing


with closing(stream_results(manager, commands)) as stream:
    for result in stream:
        consume(result)
        break
# closing 在这里调用 stream.close()。
```

`closing()` 适用于调用方确实拥有、提供 `close()` 但没有上下文协议的对象。如果对象本身已经是上下文管理器，应优先使用它的原生协议；也不要假设每一种迭代器都有 `close()` 。

18.12 常见迁移陷阱与选择标准

Java 工程师最容易误判的地方集中在以下几类：

1. **把 `with` 当作文件关键字。** 它使用通用协议，锁、事务、会话都可以参与。
2. **认为 `as` 一定绑定管理器。** 它绑定的是 `__enter__()` 返回值。
3. **只在正常路径写 `close()` 。** `return` 和异常都可能绕过这句代码。
4. **指望进入失败后自动退出。** `__enter__()` 没有成功返回时，不会调用本次 `__exit__()` 。
5. **从 `__exit__()` 返回清理结果。** 任何真值都会抑制块内异常。
6. **在 `@contextmanager` 中记录异常后漏写 `raise` 。** 这会把失败伪装成成功。
7. **按获取顺序关闭多个依赖资源。** 正确顺序通常是后进先出。
8. **借到共享客户端后主动关闭。** 能使用不代表拥有关闭权。
9. **认为 `break` 会关闭任意生成器。** 它只结束循环。
10. **依赖析构或垃圾回收释放外部资源。** 回收时机不是稳定的业务边界。

面对实际需求时，可以按下表选择：

面对实际需求时，可以按下表选择：

需求	优先写法	原因
一处局部获取与清理	`try/finally`	控制流直接可见
对象有状态、配置和多个方法	类实现 `__enter__` / `__exit__`	状态与协议放在一起
生命周期是线性过程	`@contextmanager` + 一次 `yield`	获取与清理相邻，样板少
固定少量资源	多项 `with`	进入和退出顺序清楚
运行时才知道资源数量	`ExitStack`	动态登记并逆序回收
只有 `close()` 、没有上下文协议	`closing()`	将拥有的对象适配给 `with`
传入共享资源	普通借用或 `nullcontext()`	使用但不接管关闭责任

18.13 运行本章示例

本章承接第 10 章的文件 `with` 、第 16 章的装饰器和第 17 章的生成器。目录中只有两个文件：

文件	作用
`README.md`	建立同步上下文管理器的完整心智模型
`example.py`	用可观察事件和断言验证各种生命周期路径

`example.py` 使用本地 `FakeToolSession` 模拟“分配传输层 → 握手 → 使用 → 释放”，
不访问网络，也不需要 API Key。它会验证：

- 正常退出和块内 `return` 都先清理；
- 块内异常会传给 `__exit__()` ，默认继续向外传播；
- 进入失败不会调用 `__exit__()` ，而由获取逻辑自行回滚；
- 类管理器和生成器管理器误吞异常的两种反例；
- 多项 `with` 与 `ExitStack` 的逆序退出；
- 所有者统一关闭、借用者只使用；
- 生成器在 `break` 后通过 `close()` 或 `closing()` 显式清理。

从仓库根目录运行：

从仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/18_context_managers/example.py
```

`-B` 只阻止生成 `.pyc` 缓存，不改变示例语义。成功时，最后会看到：

```
所有第 18 章示例与运行时断言均已完成。
```

现有 `java_to_python/run_all.py` 只支持前 10 章，因此应直接运行本章脚本。项目只使用标准库，没有新增运行依赖。仓库根目录的 `pyproject.toml` 要求 Python `>=3.14`。当前项目环境对应的检查命令是：

```
.venv/bin/ruff check java_to_python/chapters/18_context_managers
.venv/bin/ruff format --check java_to_python/chapters/18_context_managers
.venv/bin/pyright java_to_python/chapters/18_context_managers/example.py
```

本章验证的是同步协议、本地状态机和确定的清理顺序，不覆盖真实 HTTP/MCP/数据库客户端的握手、连接池、超时、线程安全和服务端状态。替换成第三方库后，应根据锁定版本的官方文档和集成测试重新确认关闭、重复关闭与关闭后使用的行为。

进一步核对语言和标准库语义，可以阅读 Python 3.14 官方文档：

- `with` 语句
- `contextlib.contextmanager`
- `contextlib.ExitStack`
- `generator.close()`

18.14 递进练习

练习 1：预测协议轨迹

实现一个最小 `TraceContext`，分别从块末尾、块内 `return` 和块内 `ValueError("bad input")` 离开。要求先写出预测事件，再运行核对：正常和 `return` 路径收到三个 `None`，异常路径的清理事件先于外层捕获事件。

练习 2：修复部分初始化泄漏

练习 1：预测协议轨迹

实现一个最小 `TraceContext`，分别从块末尾、块内 `return` 和块内 `ValueError("bad input")` 离开。要求先写出预测事件，再运行核对：正常和 `return` 路径收到三个 `None`，异常路径的清理事件先于外层捕获事件。

练习 2：修复部分初始化泄漏

模拟两阶段 `open_client()`：第一阶段把 `transport_open` 设为 `True`，第二阶段遇到 `"bad-token"` 时抛 `AuthenticationError`。要求失败后不进入块体、不调用 `__exit__()`，但 `transport_open` 已恢复为 `False`，原异常仍向外传播。

练习 3：修复两种异常误吞

分别修复一个无条件 `return True` 的类管理器，以及一个在 `@contextmanager` 的 `except` 中只记录却不重抛的实现。让块内固定抛 `ToolCallError("quota exceeded")`，确认两种实现都清理一次，并让外层收到原错误及原 traceback。。

练习 4：动态会话与中途失败

使用 `ExitStack` 依次进入 `"one"`、`"two"`、`"broken"`、`"never"`。让 `"broken"` 在进入时失败并自行回滚。最终应观察到：`"never"` 从未尝试，`"two"` 和 `"one"` 由栈逆序关闭，而 `"broken"` 的 `__exit__()` 没有执行。

18.15 本章小结

可以用五句话记住本章：

1. `with` 管理的是控制流边界，不只是文件。
2. `as` 接收 `__enter__()` 的返回值；进入成功后，所有离开路径都会调用 `__exit__()`。
3. 进入失败时，部分初始化必须由获取逻辑自己回滚；退出时默认不要抑制业务异常。
4. 固定资源用 `with`，动态资源用 `ExitStack`，线性生命周期可以用 `@contextmanager`。
5. 资源清理的根本问题是所有权：创建者负责关闭，借用者只在有效期内使用。

如果你能不看代码说明“第三个资源进入失败时谁清理谁”，并能解释为什么 `break` 后暂停的资源型生成器仍可能持有资源，就已经建立了同步资源生命周期的完整心智模型。

```
1     第 16 章：上下文管理器与资源生命周期。
2
3   示例使用可控的 FakeToolSession 记录获取、使用和清理事件，不访问网络，
4   也不要求 API Key。每一节都用断言校验关键生命周期，而不依赖垃圾回收时机。
5   """
6
7   # 建议按“基础模型与协议实现 -> 辅助管理器/生成器 -> 六组 demonstrate_* 实验 -> main”
8   # 的顺序阅读。所有实验把生命周期事件写入 list：代码一边打印过程，一边用同一份事件数据
9   # 做断言，由此验证正常返回、异常和提前停止等路径是否都经过了确定的清理边界。
10  #
11  # collections.abc 中的类型用于描述生成器和可迭代参数；contextlib 提供把生成器包装为
12  # 上下文管理器、动态管理多个资源以及替只有 close() 的对象补上 with 协议的工具。
13  # Enum/auto 用来建立可读的会话状态机，TracebackType 则用于精确标注 __exit__ 的第三个参数。
14  from collections.abc import Generator, Iterable
15  from contextlib import ExitStack, closing, contextmanager
16  from enum import Enum, auto
17  from types import TracebackType
18
19
20  # Enum 的成员是三个唯一的单例对象，适合表达互斥状态。auto() 让 Python 自动生成成员值；
21  # 本章只关心 NEW/OPEN/CLOSED 这些名字，不依赖底层值是几。
22  class SessionState(Enum):
23      """会话只保留本章观察生命周期所需的三个状态。"""
24
25      NEW = auto()
26      OPEN = auto()
27      CLOSED = auto()
28
29
30  # 三种异常分别标记“进入失败”“使用失败”和“生命周期结束后误用”。
31  # 像 Java 中定义专用 RuntimeException 子类一样，即使类体无需额外方法，独立类型也能让
32  # except 只捕获它真正准备处理的错误；这里继承 RuntimeError，所以都属于运行时异常。
33  class SessionInitializationError(RuntimeError):
34      """模拟握手或鉴权阶段失败。"""
35
36
```

```python
class ToolCallError(RuntimeError):
    """模拟 with 块内的业务调用失败。"""


class SessionClosedError(RuntimeError):
    """关闭后继续使用会话时给出明确错误。"""


def print_section(title: str) -> None:
    # f-string 会先计算花括号内的 title，再拼进输出；开头的 `\n` 是换行转义，
    # 让每个实验从空行开始。
    print(f"\n=== {title} ===")


def record(events: list[str], message: str) -> None:
    """事件既打印又保留，便于肉眼观察和运行时断言。"""
    # list.append 会原地修改调用方传入的同一个列表，不会创建副本。因此调用方稍后能从
    # events 中检查事件顺序；str 类型提示只供读者和静态检查器使用，不负责运行时校验。
    events.append(message)
    print(f"  事件 | {message}")


# ToolHandle 代表"可使用但不拥有"的句柄。管理器会在 __enter__ 成功后构造它，with 块和
# 借用者只能通过它查询/使用会话；真正的关闭入口仍留在 FakeToolSession 上。
class ToolHandle:
    """__enter__ 交给 with 块的借用句柄，本身不拥有关闭职责。"""

    def __init__(self, owner: FakeToolSession) -> None:
        # Python 3.14 会延迟求值注解，所以此处可以引用稍后才定义的 FakeToolSession。
        # 前导下划线表示内部属性的命名约定，不等于 Java private，运行时仍可访问。
        self._owner: FakeToolSession = owner

    # @property 把无参数方法暴露成只读属性：调用方写 handle.is_open，而不是 is_open()。
    # 它在每次访问时重新查询拥有者，没有复制状态，因此能看见 owner 后续的关闭动作。
    @property
    def is_open(self) -> bool:
```

```python
class ToolHandle:
    # @property 把无参数方法暴露成只读属性：调用方写 handle.is_open，而不是 is_open()。
    # 它在每次访问时重新查询拥有者，没有复制状态，因此能看见 owner 后续的关闭动作。
    @property
    def is_open(self) -> bool:
        # Enum 成员是单例；is 比较对象身份，正适合判断当前是否就是 OPEN 成员。
        # 不要把 is 泛化为 Java 的 equals：普通值比较通常应使用 ==。
        return self._owner.state is SessionState.OPEN

    def use(self, command: str) -> str:
        # 句柄只转发操作，由 owner 统一执行状态检查、记录事件和生成结果。
        return self._owner.use(command)


# 这个 Fake 不是在模拟真实网络细节，而是把“分配传输层 -> 握手 -> 使用 -> 释放”的状态
# 变成可以断言的数据。它既是资源所有者，也是实现 __enter__/__exit__ 协议的管理器。
class FakeToolSession:
    """可观察的同步工具会话替身。

    open() 分两步模拟传输层分配与握手。第二步失败时，open() 自己回收已经
    分配的传输层，因为 __enter__ 一旦抛异常，本次 with 不会调用 __exit__。
    """

    def __init__(
        self,
        name: str,
        events: list[str],
        *,
        fail_on_open: bool = False,
        suppress_block_error: bool = False,
    ) -> None:
        # 参数列表中的 * 表示其后的两个开关只能按关键字传入，例如 fail_on_open=True，
        # 可避免多个 bool 位置实参难以辨认。None 表示构造器只初始化实例，不返回业务值。
        self.name: str = name
        # 这里保留的是调用方列表的引用；所有 record(...) 都会写进同一个事件时间线。
        self.events: list[str] = events
        # 两个开关只用于可控地触发进入失败和异常误吞，正常会话都保持 False。
```

```
84  class FakeToolSession:
91      def __init__(
92          self,
93          name: str,
94          events: list[str],
95          *,
96          fail_on_open: bool = False,
97          suppress_block_error: bool = False,
98      ) -> None:
99          # 参数列表中的 * 表示其后的两个开关只能按关键字传入，例如 fail_on_open=True，
100         # 可避免多个 bool 位置实参难以辨认。None 表示构造器只初始化实例，不返回业务值。
101         self.name: str = name
102         # 这里保留的是调用方列表的引用；所有 record(...) 都会写进同一个事件时间线。
103         self.events: list[str] = events
104         # 两个开关只用于可控地触发进入失败和异常误吞，正常会话都保持 False。
105         self.fail_on_open: bool = fail_on_open
106         self.suppress_block_error: bool = suppress_block_error
107         # state 表示逻辑生命周期，transport_allocated 表示更底层的资源是否已分配。
108         # 两者分开，才能观察“底层已分配但握手还未完成”的部分初始化状态。
109         self.state: Literal[SessionState.NEW] = SessionState.NEW
110         self.transport_allocated = False
111         # 这两个计数器区分协议退出次数与底层资源实际释放次数，防止重复清理被掩盖。
112         self.exit_calls = 0
113         self.cleanup_count = 0
114
115     def _release_transport(self) -> None:
116         # 只有确实分配过传输层才执行释放和计数；重复调用不会把 cleanup_count 再加一。
117         # 这种幂等性让失败回滚和常规 close 即使相遇，也不会产生第二次释放。
118         if self.transport_allocated:
119             self.transport_allocated = False
120             # += 1 会把计算后的整数重新绑定给属性；int 不像 list 那样原地共享修改。
121             self.cleanup_count += 1
122             record(self.events, f"{self.name}: transport released")
123
124     def open(self) -> ToolHandle:
125         # 一个实例只能从 NEW 进入；is not 判断枚举身份，能同时拒绝重复打开和关闭后重开。
```

```
 84  class FakeToolSession:
124      def open(self) -> ToolHandle:
125          # 一个实例只能从 NEW 进入；is not 判断枚举身份，能同时拒绝重复打开和关闭后重开。
126          if self.state is not SessionState.NEW:
127              raise RuntimeError(f"{self.name}: 只能从 NEW 状态打开")
128
129          # 先记录开始，再模拟已经拿到底层传输层。此刻尚未握手成功，所以不能提前设为 OPEN。
130          record(self.events, f"{self.name}: open started")
131          self.transport_allocated = True
132          record(self.events, f"{self.name}: transport allocated")
133
134          # try 覆盖握手及句柄构造：其中任一步失败，都要撤销上面已经完成的部分初始化。
135          try:
136              if self.fail_on_open:
137                  record(self.events, f"{self.name}: handshake failed")
138                  raise SessionInitializationError(f"{self.name}: 初始化失败")
139              # 只有握手通过后才发布 OPEN 状态，并返回一个借用句柄给 with 的 as 目标。
140              # 返回 ToolHandle(self)，也证明管理器对象和块内拿到的值不必是同一个对象。
141              self.state: Literal[SessionState.OPEN] = SessionState.OPEN
142              record(self.events, f"{self.name}: open completed")
143              return ToolHandle(self)
144          except BaseException:
145              # __enter__ 尚未成功，with 不会替我们调用 __exit__；部分初始化必须自清理。
146              # 捕获 BaseException 是为了连 KeyboardInterrupt 等退出信号也先完成资源回滚；
147              # 末尾的裸 raise 会原样重抛当前异常，并保留原 traceback，不会把失败转成成功。
148              record(self.events, f"{self.name}: open rolls back partial state")
149              self._release_transport()
150              self.state: Literal[SessionState.CLOSED] = SessionState.CLOSED
151              raise
152
153      def use(self, command: str) -> str:
154          # 每次使用都先检查生命周期，使关闭后误用立即暴露，而不是悄悄访问失效资源。
155          if self.state is not SessionState.OPEN:
156              raise SessionClosedError(f"{self.name}: 会话未打开或已关闭")
157
158          record(self.events, f"{self.name}: use {command}")
```

```
 84  class FakeToolSession:
152
153      def use(self, command: str) -> str:
154          # 每次使用都先检查生命周期，使关闭后误用立即暴露，而不是悄悄访问失效资源。
155          if self.state is not SessionState.OPEN:
156              raise SessionClosedError(f"{self.name}: 会话未打开或已关闭")
157
158          record(self.events, f"{self.name}: use {command}")
159          # 特殊命令只为稳定触发 with 块内部的业务异常，供后面的异常路径实验观察。
160          if command == "business-error":
161              raise ToolCallError(f"{self.name}: 工具调用失败")
162          # 普通命令返回字符串；资源状态保持 OPEN，可以在同一生命周期中继续复用。
163          return f"{self.name} -> {command} 完成"
164
165      def close(self) -> None:
166          """清理设计为幂等，避免重复关闭造成第二个故障。"""
167          # 已关闭时只记录并返回。没有显式返回值的 Python 函数会返回 None。
168          if self.state is SessionState.CLOSED:
169              record(self.events, f"{self.name}: close skipped (already closed)")
170              return
171
172          # 先记录高层 close，再释放底层传输层，最后把逻辑状态推进到 CLOSED。
173          record(self.events, f"{self.name}: close")
174          self._release_transport()
175          self.state: Literal[SessionState.CLOSED] = SessionState.CLOSED
176
177      # with manager 会先调用 __enter__；这里复用 open 的状态检查和失败回滚，并把它返回的
178      # ToolHandle 继续交给 as 目标。Java try-with-resources 通常暴露资源本身，Python 则允许
179      # __enter__ 返回另一个受限视图。
180      def __enter__(self) -> ToolHandle:
181          return self.open()
182
183      # __enter__ 成功后，无论块体正常结束、return 还是抛异常，with 都会调用一次 __exit__。
184      # `T | None` 表示参数可以是 T 或 None：正常离开时三个参数全是 None，异常离开时分别是
185      # 异常类、异常实例和 traceback。类型提示不改变运行时协议。
186      def __exit__(
187          self,
```

```python
class FakeToolSession:

    # __enter__ 成功后，无论块体正常结束、return 还是抛异常，with 都会调用一次 __exit__。
    # `T | None` 表示参数可以是 T 或 None：正常离开时三个参数全是 None，异常离开时分别是
    # 异常类、异常实例和 traceback。类型提示不改变运行时协议。
    def __exit__(
        self,
        exc_type: type[BaseException] | None,
        exc_value: BaseException | None,
        traceback: TracebackType | None,
    ) -> bool:
        # 先把调用次数加一，再把异常对象转换为稳定、可读的事件文本。
        self.exit_calls += 1
        # `A if 条件 else B` 是条件表达式；这里避免对 None 读取 __name__ 或调用 str 之外
        # 的属性。异常类本身提供 __name__，异常实例则由 str(...) 给出消息。
        exception_name: str = "None" if exc_type is None else exc_type.__name__
        exception_text: str = "None" if exc_value is None else str(exc_value)
        has_traceback: bool = traceback is not None
        # 相邻的两个 f-string 会在语法层面自动拼接为一个字符串，用于控制源码行宽；
        # record 最终收到的仍是一条完整事件。
        record(
            self.events,
            f"{self.name}: __exit__(type={exception_name}, "
            f"value={exception_text}, traceback={has_traceback})",
        )
        # 无论块内是否失败都先关闭资源，然后才通过返回值决定原异常是否继续传播。
        self.close()

        # 正常资源管理器通常返回 False（或 None），让块内异常继续传播。
        # 这个开关只为受控演示“真值会抑制异常”，不应成为会话的默认行为。
        return self.suppress_block_error


def return_from_with(manager: FakeToolSession) -> str:
    """return 先记录待返回值，但调用方要等 __exit__ 完成后才能收到它。"""
    # return 出现在 with 块内并不会跳过协议退出。Python 先准备好 "返回成功"，调用
    # manager.__exit__(None, None, None) 完成清理后，调用方才能真正拿到该字符串。
```

```python
def return_from_with(manager: FakeToolSession) -> str:
    """return 先记录待返回值，但调用方要等 __exit__ 完成后才能收到它。"""
    # return 出现在 with 块内并不会跳过协议退出。Python 先准备好 "返回成功"，调用
    # manager.__exit__(None, None, None) 完成清理后，调用方才能真正拿到该字符串。
    with manager as handle:
        handle.use("prepare-return")
        record(manager.events, "函数: 执行 return")
        return "返回成功"

    # 只有 return 之前出现异常且 __exit__ 错误地将它抑制，才可能落到这里。
    raise AssertionError("return 前的异常被上下文管理器意外抑制")


def use_with_try_finally(events: list[str], *, fail: bool) -> FakeToolSession:
    """先展示不依赖协议的获取/使用/清理骨架。"""
    # 资源在进入 try 之前已成功取得；若 open 自己失败，它会自行回滚，也不会执行这里的
    # finally。成功后 manager 是所有者，handle 只是供 try 块使用的借用句柄。
    manager = FakeToolSession("manual", events)
    handle: ToolHandle = manager.open()
    try:
        handle.use("manual-call")
        # fail=True 用来证明异常离开 try 时仍会先进入 finally；fail=False 则会准备返回
        # manager，但同样要等 finally 的 close 完成后才真正返回。
        if fail:
            raise ToolCallError("manual: try 块失败")
        return manager
    finally:
        # finally 不负责决定是否吞掉异常，只保证离开使用区间时清理。
        manager.close()


# 装饰器在函数定义完成时执行，效果近似
# managed_tool_session = contextmanager(managed_tool_session)。装饰后的调用返回上下文管理器，
# 不是直接返回 ToolHandle；Generator[ToolHandle] 描述这段生成器会 yield 的值类型。
@contextmanager
def managed_tool_session(name: str, events: list[str]) -> Generator[ToolHandle]:
    """用恰好一次 yield 表达同一个获取/使用/清理骨架。"""
```

```
244
245 # 装饰器在函数定义完成时执行，效果近似
246 # managed_tool_session = contextmanager(managed_tool_session)。装饰后的调用返回上下文管理器，
247 # 不是直接返回 ToolHandle；Generator[ToolHandle] 描述这段生成器会 yield 的值类型。
248 @contextmanager
249 def managed_tool_session(name: str, events: list[str]) -> Generator[ToolHandle]:
250     """用恰好一次 yield 表达同一个获取/使用/清理骨架。"""
251     # with 进入装饰器生成的管理器时，函数体才开始运行：先创建并打开真实管理对象。
252     manager = FakeToolSession(name, events)
253     handle: ToolHandle = manager.open()
254     record(events, f"{name}: contextmanager yield 前")
255     try:
256         # yield 暂停当前函数，并把 handle 交给 with 的 as 目标；暂停期间执行的是 with 块。
257         yield handle
258     except BaseException as error:
259         # with 块内的异常会被注入到暂停的 yield 位置；这里只记录，所以必须重抛。
260         # type(error).__name__ 先取异常实例的类，再取类名；裸 raise 保留原异常和 traceback。
261         record(events, f"{name}: yield 收到 {type(error).__name__}")
262         raise
263     finally:
264         # 正常退出和异常退出都会执行 finally，这一段对应类管理器的 __exit__ 清理责任。
265         record(events, f"{name}: contextmanager finally")
266         manager.close()
267
268
269 # 这个装饰器用法与上面相同，但函数体故意漏掉重抛，用来展示一个容易被 Java 经验忽略的
270 # 差异：块内异常会回到 yield 处，若生成器随后正常结束，contextmanager 会把它视为已处理。
271 @contextmanager
272 def swallowing_tool_session(name: str, events: list[str]) -> Generator[ToolHandle]:
273     """教学反例：记录异常后忘记 raise，会把故障伪装成成功。"""
274     manager = FakeToolSession(name, events)
275     handle: ToolHandle = manager.open()
276     try:
277         yield handle
278     except ToolCallError as error:
279         # as error 把当前 ToolCallError 实例绑定为局部变量，f-string 会读取它的错误消息。
280         record(events, f"{name}: 只记录但误吞 {error}")
```

```python

# 这个装饰器用法与上面相同，但函数体故意漏掉重抛，用来展示一个容易被 Java 经验忽略的
# 差异：块内异常会回到 yield 处，若生成器随后正常结束，contextmanager 会把它视为已处理。
@contextmanager
def swallowing_tool_session(name: str, events: list[str]) -> Generator[ToolHandle]:
    """教学反例：记录异常后忘记 raise，会把故障伪装成成功。"""
    manager = FakeToolSession(name, events)
    handle: ToolHandle = manager.open()
    try:
        yield handle
    except ToolCallError as error:
        # as error 把当前 ToolCallError 实例绑定为局部变量，f-string 会读取它的错误消息。
        record(events, f"{name}: 只记录但误吞 {error}")
        # 教学反例：这里缺少 raise。生成器正常结束后，异常会被抑制。
    finally:
        manager.close()


# @contextmanager 要求每次生命周期恰好到达一次 yield。下面把 yield 放进永假的分支，
# Python 仍把该函数编译为生成器函数，但 with 进入时它会直接耗尽，因此触发 RuntimeError。
@contextmanager
def context_without_yield() -> Generator[None]:
    """教学反例：保留生成器形态，但进入阶段没有实际 yield。"""
    if False:
        yield


# 第一次 yield 可以正常完成进入并把 "first" 绑定给 as；退出阶段生成器恢复后若又 yield，
# contextmanager 发现生成器没有按约定结束，也会抛 RuntimeError。
@contextmanager
def context_with_two_yields(events: list[str]) -> Generator[str]:
    """教学反例：with 块结束后又 yield，违反恰好一次的约定。"""
    yield "first"
    record(events, "two-yields: 准备错误地执行第二次 yield")
    yield "second"


```

```python
def borrower_run_tool(handle: ToolHandle, command: str, events: list[str]) -> str:
    """借用者只使用传入句柄，不关闭不属于自己的会话。"""
    # 该函数没有创建 handle，也没有接收所有权，因此只负责一次 use 并返回结果。
    # 这类似方法临时接收由外层 try-with-resources 管理的 Java 资源：能调用不等于应关闭。
    record(events, f"borrower: 开始 {command}")
    result: str = handle.use(command)
    record(events, f"borrower: 完成 {command}，不关闭共享会话")
    return result


def stream_tool_results(
    manager: FakeToolSession,
    commands: Iterable[str],
    events: list[str],
) -> Generator[str]:
    """资源型生成器：首次消费时打开，结束或 close() 时退出内部 with。"""
    # 调用生成器函数只创建 generator 对象，不立刻执行这里。第一次 next()（通常由 for
    # 隐式调用）才记录开始并进入 manager，因此创建流之后会话仍保持 NEW。
    record(events, "stream: 生成器函数体开始")
    with manager as handle:
        # Iterable[str] 允许 tuple、list 或其他逐项产生字符串的对象。每轮 command 绑定
        # 下一条命令，use 的返回字符串经 yield 交给消费者，然后生成器暂停在这一行。
        for command in commands:
            record(events, f"stream: 准备产出 {command}")
            yield handle.use(command)
    # 只有 commands 自然耗尽时才会运行到这里；若外部调用 close()，内部 with 的清理仍会
    # 执行，但 GeneratorExit 会让控制流离开生成器，不会记录“自然耗尽”。
    record(events, "stream: 自然耗尽")


def demonstrate_with_protocol() -> None:
    # 第一组实验先走最普通的成功路径，观察管理器、as 绑定值和协议退出。
    print_section("1. with 协议、as 绑定值与 return")
    # `list[str]` 是类型提示；空列表仍是普通可变对象。manager 保存对它的引用，后续所有
    # open/use/close 事件都会按发生顺序追加进来。
    events: list[str] = []
    manager = FakeToolSession("normal", events)
```

```python

def demonstrate_with_protocol() -> None:
    # 第一组实验先走最普通的成功路径，观察管理器、as 绑定值和协议退出。
    print_section("1. with 协议、as 绑定值与 return")
    # `list[str]` 是类型提示；空列表仍是普通可变对象。manager 保存对它的引用，后续所有
    # open/use/close 事件都会按发生顺序追加进来。
    events: list[str] = []
    manager = FakeToolSession("normal", events)

    # 执行顺序是 manager.__enter__() -> 把返回的 ToolHandle 绑定给 handle -> 块体 ->
    # manager.__exit__(None, None, None)。只有 __enter__ 成功，退出责任才正式建立。
    with manager as handle:
        # `is not` 比较对象身份，结果为 True，证明 as 绑定的是借用句柄而非 manager。
        print("  as 绑定 ToolHandle，而不是管理器本身:", handle is not manager)
        print("  调用结果:", handle.use("search"))

    # 正常退出的最后三条事件依次是 __exit__、close、transport released，所以 events[-3]
    # 用负索引从列表末尾倒数第三项取出退出事件。assert 失败才抛 AssertionError；通过时无输出。
    print("  正常退出后三个异常参数均为空:", "type=None" in events[-3])
    print("  最终状态:", manager.state.name)
    assert manager.state is SessionState.CLOSED
    assert manager.exit_calls == 1

    # 第二条成功路径把 return 放进另一个函数的 with 块。分别记录“开始返回”“close”和
    # “调用方收到结果”，再比较它们在列表中的位置，验证清理先于结果交付。
    return_events: list[str] = []
    return_manager = FakeToolSession("return", return_events)
    result: str = return_from_with(return_manager)
    record(return_events, f"调用方: 收到 {result}")
    # list.index 返回指定事件第一次出现的零基下标；close 下标更小就表示它发生得更早。
    close_index: int = return_events.index("return: close")
    receive_index: int = return_events.index("调用方: 收到 返回成功")
    print("  return 的清理先于调用方收到结果:", close_index < receive_index)
    assert close_index < receive_index


def demonstrate_exception_paths() -> None:
```

```
369
370 def demonstrate_exception_paths() -> None:
371     print_section("2. 块内异常、初始化失败与误吞异常")
372     # 场景一：__enter__ 已成功，错误来自 with 块。__exit__ 应先收到异常三元组并关闭，
373     # 因为默认返回 False，原 ToolCallError 随后继续传播到外层 except。
374     block_events: list[str] = []
375     block_manager = FakeToolSession("block-error", block_events)
376     try:
377         with block_manager as handle:
378             handle.use("business-error")
379     except ToolCallError as error:
380         print("  外层仍收到块内异常:", error)
381
382     # 生成器表达式逐条筛选包含 "__exit__(" 的事件，next(...) 只取第一个匹配项。
383     # 本场景确定存在该事件；若不存在，next 会抛 StopIteration，也会让实验失败。
384     exit_event: str = next(event for event in block_events if "__exit__(" in event)
385     print("  __exit__ 收到异常三元组:", exit_event)
386     print("  异常路径仍已关闭:", block_manager.state is SessionState.CLOSED)
387     assert "type=ToolCallError" in exit_event
388     assert "traceback=True" in exit_event
389     assert block_manager.state is SessionState.CLOSED
390
391     # 场景二：让 open 在分配 transport 后、返回句柄前失败。body_started 是哨兵变量：
392     # 只有 with 块真的执行过才会变成 True，据此证明 __enter__ 失败会阻止块体开始。
393     init_events: list[str] = []
394     init_manager = FakeToolSession("init-error", init_events, fail_on_open=True)
395     body_started = False
396     try:
397         with init_manager:
398             body_started = True
399     except SessionInitializationError as error:
400         print("  捕获初始化错误:", error)
401
402     # 此时协议尚未建立退出责任，所以 exit_calls 必须为 0；cleanup_count 为 1 则证明
403     # open 自己回滚了已经分配的 transport。两条规则必须同时成立才没有资源泄漏。
404     print("  __enter__ 失败后块未运行:", not body_started)
405     print("  with 没有调用 __exit__:", init_manager.exit_calls == 0)
```

```
370  def demonstrate_exception_paths() -> None:
402      # 此时协议尚未建立退出责任，所以 exit_calls 必须为 0；cleanup_count 为 1 则证明
403      # open 自己回滚了已经分配的 transport。两条规则必须同时成立才没有资源泄漏。
404      print("  __enter__ 失败后块未运行:", not body_started)
405      print("  with 没有调用 __exit__:", init_manager.exit_calls == 0)
406      print("  open 自行回收部分资源:", init_manager.cleanup_count == 1)
407      assert not body_started
408      assert init_manager.exit_calls == 0
409      assert init_manager.cleanup_count == 1
410      assert init_manager.state is SessionState.CLOSED
411
412      # 场景三是故意保留的反例：管理器会清理资源，但 __exit__ 返回 True，使块内异常被吞掉。
413      # 若异常正常传播，赋值 reached_after_with=True 根本不会执行；它最终为 True 就是故障证据。
414      suppression_events: list[str] = []
415      bad_manager = FakeToolSession(
416          "suppress-error",
417          suppression_events,
418          suppress_block_error=True,
419      )
420      reached_after_with = False
421      with bad_manager as handle:
422          handle.use("business-error")
423      reached_after_with = True
424      print("  教学反例：返回真值后故障被隐藏:", reached_after_with)
425      assert reached_after_with
426      assert bad_manager.state is SessionState.CLOSED
427
428
429  def demonstrate_finally_and_contextmanager() -> None:
430      print_section("3. 从 try/finally 到 @contextmanager")
431      # 先用最基础的 try/finally 骨架。fail=True 让 try 抛错，finally 仍先关闭资源；
432      # finally 没有 return 或新异常，所以原 ToolCallError 保持不变地交给外层。
433      manual_events: list[str] = []
434      try:
435          use_with_try_finally(manual_events, fail=True)
436      except ToolCallError as error:
437          print("  try/finally 保留原异常:", error)
```

```python
def demonstrate_finally_and_contextmanager() -> None:
    print_section("3. 从 try/finally 到 @contextmanager")
    # 先用最基础的 try/finally 骨架。fail=True 让 try 抛错，finally 仍先关闭资源；
    # finally 没有 return 或新异常，所以原 ToolCallError 保持不变地交给外层。
    manual_events: list[str] = []
    try:
        use_with_try_finally(manual_events, fail=True)
    except ToolCallError as error:
        print("  try/finally 保留原异常:", error)
    print("  try/finally 已执行 close:", "manual: close" in manual_events)
    assert "manual: close" in manual_events

    # 再验证生成器管理器的对应路径：块内错误会在 yield 暂停点重新出现，except 记录它，
    # 裸 raise 重抛，finally 关闭，最后同一个异常才到达这里的外层 except。
    generator_events: list[str] = []
    try:
        with managed_tool_session("generated", generator_events) as handle:
            handle.use("business-error")
    except ToolCallError as error:
        print("  yield 处记录后重抛，外层仍收到:", error)

    # 两次 index 得到关键事件的位置，用 `<` 验证异常先回到 yield、随后才进入 finally。
    received_index: int = generator_events.index("generated: yield 收到 ToolCallError")
    finally_index: int = generator_events.index("generated: contextmanager finally")
    print("  异常先回到 yield，再执行 finally:", received_index < finally_index)
    assert received_index < finally_index
    assert "generated: close" in generator_events

    # 对照反例没有裸 raise：contextmanager 看到生成器处理异常后正常结束，就抑制块内错误。
    # 因而 with 后的赋值可以执行，这不是成功，而是错误被错误地隐藏了。
    swallowed_events: list[str] = []
    continued = False
    with swallowing_tool_session("forgot-raise", swallowed_events) as handle:
        handle.use("business-error")
    continued = True
    print("  教学反例：contextmanager 漏写 raise 后继续运行:", continued)
```

```
429  def demonstrate_finally_and_contextmanager() -> None:
461      with swallowing_tool_session("forgot-raise", swallowed_events) as handle:
462          handle.use("business-error")
463      continued = True
464      print("  教学反例: contextmanager 漏写 raise 后继续运行:", continued)
465      assert continued
466
467      # 零次 yield 会在进入阶段被包装器拒绝，块内的 pass 不会执行任何操作。
468      try:
469          with context_without_yield():
470              pass
471      except RuntimeError as error:
472          print("  零次 yield 被拒绝:", error)
473
474      # 两次 yield 的第一次值能绑定给 value；离开块时包装器要求生成器结束，却遇到第二次
475      # yield，于是抛 RuntimeError。事件列表精确证明恢复后确实走到了第二次 yield 之前。
476      two_yield_events: list[str] = []
477      try:
478          with context_with_two_yields(two_yield_events) as value:
479              print("  第一次 yield 绑定值:", value)
480      except RuntimeError as error:
481          print("  两次 yield 被拒绝:", error)
482      assert two_yield_events == ["two-yields: 准备错误地执行第二次 yield"]
483
484
485  def demonstrate_multiple_resources() -> None:
486      print_section("4. 多资源逆序退出与 ExitStack")
487      # 三个管理器共享同一事件列表，便于从一条时间线观察固定数量资源的进入与退出顺序。
488      static_events: list[str] = []
489      first = FakeToolSession("A", static_events)
490      second = FakeToolSession("B", static_events)
491      third = FakeToolSession("C", static_events)
492
493      # 逗号分隔的多项 with 等价于三层嵌套：按 A、B、C 进入，按 C、B、A 退出。
494      # 后进入的资源可能依赖先进入的资源，因此像栈一样后进先出。
495      with first as first_handle, second as second_handle, third as third_handle:
```

```
485  def demonstrate_multiple_resources() -> None:
493      # 逗号分隔的多项 with 等价于三层嵌套：按 A、B、C 进入，按 C、B、A 退出。
494      # 后进入的资源可能依赖先进入的资源，因此像栈一样后进先出。
495      with first as first_handle, second as second_handle, third as third_handle:
496          first_handle.use("one")
497          second_handle.use("two")
498          third_handle.use("three")
499
500      # 列表推导式从 static_events 只保留以 ": close" 结尾的事件。split(..., maxsplit=1)
501      # 最多切一次，再取 [0]，所以 "C: close" 会提取出资源名 "C"。
502      close_order: list[str] = [
503          event.split(":", maxsplit=1)[0] for event in static_events if event.endswith(": close")
504      ]
505      # 字典推导式逐个读取三个 manager，产生 {名称: 最终状态名}，便于一次检查全体资源。
506      final_states: dict[str, str] = {manager.name: manager.state.name for manager in (first, second, third)}
507      print("  声明顺序: A -> B -> C")
508      print("  退出顺序:", " -> ".join(close_order))
509      print("  所有最终状态:", final_states)
510      assert close_order == ["C", "B", "A"]
511      # dict.values() 取得状态值视图，set(...) 去重；结果只有 {"CLOSED"} 表示三者全已关闭。
512      assert set(final_states.values()) == {"CLOSED"}
513
514      # 当资源数量来自运行时列表，源码无法预先写死多项 with。ExitStack 让循环每成功进入一个
515      # manager 就登记一个 __exit__，离开 stack 的 with 时再统一逆序调用。
516      runtime_names: list[str] = ["runtime-1", "runtime-2", "runtime-3"]
517      stack_events: list[str] = []
518      runtime_managers: list[FakeToolSession] = []
519      with ExitStack() as stack:
520          for name in runtime_names:
521              # 每轮创建的 manager 都写同一事件列表；append 保存对象供退出后检查状态。
522              manager = FakeToolSession(name, stack_events)
523              runtime_managers.append(manager)
524              # enter_context 会立即调用 manager.__enter__，将成功进入后的退出方法压栈，
525              # 并把 __enter__ 的返回值交给 handle，效果相当于动态增加一层 with。
526              handle: ToolHandle = stack.enter_context(manager)
527              handle.use("dynamic-call")
528
```

```
485  def demonstrate_multiple_resources() -> None:
528
529      # reversed 返回按相反方向遍历 runtime_names 的迭代器；list(...) 将它收集成列表，
530      # 与实际 close_order 比较，验证动态栈同样后进先出。
531      runtime_close_order: list[str] = [
532          event.split(":", maxsplit=1)[0] for event in stack_events if event.endswith(": close")
533      ]
534      print("  ExitStack 动态退出顺序:", " -> ".join(runtime_close_order))
535      assert runtime_close_order == list(reversed(runtime_names))
536
537      # 最后一组让第三个资源在 __enter__ 中失败。attempted 包含所有已经尝试构造的管理器，
538      # 因而可以同时检查前两个成功资源和第三个半初始化资源分别由谁清理。
539      failure_events: list[str] = []
540      attempted: list[FakeToolSession] = []
541      try:
542          with ExitStack() as stack:
543              for name in ("ready-1", "ready-2", "fails-3"):
544                  # 比较表达式只在 name 为 "fails-3" 时得到 True，作为关键字参数触发失败。
545                  manager = FakeToolSession(
546                      name,
547                      failure_events,
548                      fail_on_open=name == "fails-3",
549                  )
550                  attempted.append(manager)
551                  # 第三个 enter_context 抛错后，循环立即中止。ExitStack 离开时清理已经入栈的
552                  # ready-2、ready-1；失败对象没入栈，依靠自己的 open 回滚。
553                  stack.enter_context(manager)
554      except SessionInitializationError as error:
555          print("  后续获取失败:", error)
556
557      # [-1] 取得最后尝试的 fails-3。列表推导式随后验证三个 exit_calls 恰为 [1, 1, 0]：
558      # 两个成功进入者各由栈退出一次，进入失败者不调用 __exit__。
559      failed_manager: FakeToolSession = attempted[-1]
560      failure_states: dict[str, str] = {manager.name: manager.state.name for manager in attempted}
561      print("  已成功进入的资源由 ExitStack 回收:", failure_states)
562      print("  失败资源的 __exit__ 调用次数:", failed_manager.exit_calls)
563      assert set(failure_states.values()) == {"CLOSED"}
```

```
485  def demonstrate_multiple_resources() -> None:
561      print("  已成功进入的资源由 ExitStack 回收:", failure_states)
562      print("  失败资源的 __exit__ 调用次数:", failed_manager.exit_calls)
563      assert set(failure_states.values()) == {"CLOSED"}
564      assert [manager.exit_calls for manager in attempted] == [1, 1, 0]
565      assert failed_manager.cleanup_count == 1
566
567
568  def demonstrate_ownership() -> None:
569      print_section("5. 生命周期的拥有者与借用者")
570      # owner 创建并持有会话，with 划定它唯一的打开区间；shared_handle 只在该区间借给函数。
571      events: list[str] = []
572      owner = FakeToolSession("shared", events)
573
574      with owner as shared_handle:
575          # 两个借用函数顺序复用同一句柄，都不 close；因此第一个返回后第二个仍能正常调用。
576          first_result: str = borrower_run_tool(shared_handle, "tool-A", events)
577          second_result: str = borrower_run_tool(shared_handle, "tool-B", events)
578          print("  两个借用者之间会话仍打开:", shared_handle.is_open)
579          print("  借用结果:", first_result, "/", second_result)
580
581      # Python 的名字离开 with 后仍存在，shared_handle 仍指向原对象；with 管理的是资源
582      # 生命周期，不是变量作用域。此时 owner 已 CLOSED，所以再次 use 会得到明确异常。
583      print("  所有者离开 with 后关闭:", owner.state is SessionState.CLOSED)
584      try:
585          shared_handle.use("too-late")
586      except SessionClosedError as error:
587          print("  关闭后的句柄不能继续使用:", error)
588      # 实际底层释放只有一次，证明借用者没有提前或重复接管关闭责任。
589      assert owner.cleanup_count == 1
590
591
592  def demonstrate_generator_cleanup() -> None:
593      print_section("6. 生成器提前停止需要显式清理")
594      # 调用 stream_tool_results 只返回惰性的 generator，函数体尚未执行，manager 仍是 NEW。
595      manual_events: list[str] = []
```

```
591
592 def demonstrate_generator_cleanup() -> None:
593     print_section("6. 生成器提前停止需要显式清理")
594     # 调用 stream_tool_results 只返回惰性的 generator，函数体尚未执行，manager 仍是 NEW。
595     manual_events: list[str] = []
596     manual_manager = FakeToolSession("manual-stream", manual_events)
597     stream: Generator[str, None, None] = stream_tool_results(manual_manager, ("chunk-1", "chunk-2"), manual_e
598     print("  创建生成器后会话状态:", manual_manager.state.name)
599
600     # for 第一次取值时才进入生成器内部的 with 并打开会话。break 只退出 for，不会自动调用
601     # 任意 iterator 的 close；变量 stream 仍引用暂停在 yield 处的生成器，所以资源仍 OPEN。
602     for result in stream:
603         print("  消费者只取一项:", result)
604         break
605
606     print("  break 后、显式 close 前仍打开:", manual_manager.state is SessionState.OPEN)
607     # generator.close() 会在暂停的 yield 位置触发 GeneratorExit，使内部 with 执行 __exit__。
608     # 这给提前停止建立了确定的清理时机，而不是等待垃圾回收。
609     stream.close()
610     print("  generator.close() 后已关闭:", manual_manager.state is SessionState.CLOSED)
611     assert manual_manager.state is SessionState.CLOSED
612
613     # closing(...) 把"有 close() 但自身不作为上下文管理器使用"的 generator 适配给 with。
614     # 调用方拥有 managed_stream，因此适合在离开消费边界时由 closing 统一调用 close()。
615     closing_events: list[str] = []
616     closing_manager = FakeToolSession("closing-stream", closing_events)
617     managed_stream: Generator[str, None, None] = stream_tool_results(
618         closing_manager,
619         ("chunk-1", "chunk-2"),
620         closing_events,
621     )
622     with closing(managed_stream) as results:
623         # closing.__enter__ 返回原对象本身，所以 results 和 managed_stream 指向同一生成器。
624         for result in results:
625             print("  closing 管理的消费者只取一项:", result)
626             break
627
```

```
592  def demonstrate_generator_cleanup() -> None:
627
628      # 即便循环只消费一项，离开 closing 的 with 也会调用 managed_stream.close()，继而退出
629      # 生成器内部的 with 并关闭 closing_manager。
630      print("  离开 closing 后已关闭:", closing_manager.state is SessionState.CLOSED)
631      assert closing_manager.state is SessionState.CLOSED
632
633
634  def main() -> None:
635      # 六组实验彼此创建独立 Fake 和事件列表，按课程顺序运行；任何 assert 失败都会立即指出
636      # 对应生命周期假设不成立，全部通过后才打印最终完成消息。
637      demonstrate_with_protocol()
638      demonstrate_exception_paths()
639      demonstrate_finally_and_contextmanager()
640      demonstrate_multiple_resources()
641      demonstrate_ownership()
642      demonstrate_generator_cleanup()
643      print("\n所有第 18 章示例与运行时断言均已完成。")
644
645
646  # 导入此文件时，Python 会定义上面的类和函数，但不会自动运行实验；只有把文件作为脚本
647  # 直接执行时，内置变量 __name__ 才等于 "__main__"。这相当于显式的程序入口保护，避免
648  # 其他模块为了复用 FakeToolSession 而 import 时产生打印和断言副作用。
649  if __name__ == "__main__":
650      main()
651
```